

PROJECT REPORT

AWS AI SECURITY HARDENING LAB

Automated Security Assessment, Detection and Remediation of AWS AI Workloads

Submitted by
K. Sthuthi Nayak

Abstract

Cloud-based Artificial Intelligence (AI) applications depend on multiple AWS services such as Amazon EC2, Amazon S3, AWS Identity and Access Management (IAM), AWS Secrets Manager, AWS CloudTrail and Amazon VPC. Misconfiguration of these services can introduce serious security risks, including excessive IAM privileges, publicly accessible resources, unencrypted storage, insecure network exposure, hard-coded credentials and inadequate logging.

This project presents an AWS AI Security Hardening Lab designed to demonstrate the complete security lifecycle of an AI-oriented AWS workload. The environment was initially configured with intentionally vulnerable security settings. A Python-based security scanner using Boto3 was developed to automatically discover project resources and evaluate them against predefined security rules.

The scanner evaluates 14 security controls across IAM, Amazon S3, network security, secrets management, CloudTrail, AI workload security and storage encryption. Vulnerabilities identified during the assessment were subsequently remediated by applying least-privilege IAM policies, restricting network exposure, enabling secure S3 configurations, removing hard-coded credentials, validating CloudTrail logging, enforcing encrypted storage and applying AI application-level security controls.

After remediation, the final security assessment produced 14/14 PASS with zero failed checks, demonstrating that the implemented controls successfully addressed the vulnerabilities targeted by the laboratory.

Keywords

AWS Security, AI Security, Cloud Security, IAM, S3, EC2, Boto3, Security Hardening, DevSecOps, Least Privilege.

Executive Summary

This project implements a controlled AWS security hardening laboratory for an AI-oriented workload. The environment was intentionally configured with security weaknesses, assessed using an automated Python/Boto3 scanner, remediated using AWS security controls, and independently re-assessed.

The assessment covers 14 security controls across IAM, S3, networking, secrets management, CloudTrail, Generative-AI application security and EBS encryption. The final assessment produced 14 PASS and 0 FAIL, confirming that the defined controls were satisfied at the time of validation.

Table of Contents

Abstract.....	2
Keywords.....	2
Executive Summary.....	2
Table of Contents.....	3
1. Introduction.....	6
1.1 Project Contributions	6
1.2 Project Success Criteria	6
1.3 Security Engineering Approach.....	6
1.4 Assumptions.....	6
1.5 Terminology.....	6
2. Problem Statement.....	7
3. Objectives	7
4. Scope of the Project	8
5. Background.....	8
5.1 Cloud Security	8
5.2 Least Privilege	8
5.3 Network Security	8
5.4 Storage Encryption	8
5.5 Secrets Management	8
6. System Architecture.....	8
7. Technology Stack	9
8. AWS Infrastructure.....	9
8.1 EC2	9
8.2 Amazon S3.....	9
8.3 IAM.....	9
8.4 AWS Secrets Manager.....	9
8.5 CloudTrail.....	10
9. Threat Model.....	10
10. Security Assessment Checks.....	10
11. Initial Vulnerable Configuration.....	10
11.1 Excessive IAM Permissions	10
11.2 Public SSH Access.....	10
11.3 Public EC2 Exposure	11

11.4 Hard-Coded Credentials.....	11
11.5 AI Workload Over-Permission	11
12. Detection Methodology	11
12.1 Security Assessment Logic	11
12.2 Security Check Design Principles.....	11
12.3 Expected Secure State.....	11
12.4 Vulnerability-to-Control Mapping.....	11
12.5 Risk Prioritization	11
13. Remediation and Hardening	12
13.1 IAM Hardening.....	12
13.2 Network Hardening.....	12
13.3 S3 Hardening	12
13.4 Secrets Management	12
13.5 EBS Encryption	12
13.6 CloudTrail.....	12
13.7 AI Application Controls.....	12
13.8 Remediation Verification Strategy	13
14. Scanner Implementation	13
14.9 Evidence and Reporting.....	13
14.10 Security Scanner Execution	13
14.1 Resource Discovery	13
14.2 IAM Checks.....	13
14.3 S3 Checks	13
14.4 Network Checks.....	14
14.5 Secrets Check.....	14
14.6 CloudTrail Check.....	14
14.7 Encryption Check	14
14.8 AI Security Checks	14
15. Cross-Platform Implementation.....	14
15.1 Why Boto3 Was Used.....	14
15.2 Authentication and Credential Handling.....	14
15.3 Cross-Platform Considerations	14
15.4 Operational Workflow	14
15.5 Example Vulnerable Configuration	15
15.6 Hardened Application Configuration.....	15

15.7 Validation Philosophy.....	15
16. Final Security Validation.....	15
Final result:	15
17. Before-and-After Comparison	16
18. Security Analysis	16
19. Limitations	16
20. Future Enhancements.....	17
21. Conclusion	17
22. References.....	17
23. Appendix.....	18
Appendix A – Project Directory	18
Appendix B – Important Commands	18
Appendix C – Recommended Screenshots	18
Appendix D – Report Reading Note.....	20
Appendix E – Detailed Security Check Catalogue	21
Appendix G – Suggested Figure and Screenshot Placement	23
Appendix H – Reproducibility Checklist.....	24
APPENDIX G – IMPLEMENTATION AND SCAN EVIDENCE	25

Evidence — Recorded reason/result supporting a security check.

Hardening — Reducing unnecessary exposure and privilege.

Resource Discovery — Finding relevant AWS resources dynamically.

Boto3 — Python SDK for AWS.

Defense in Depth — Use multiple independent controls.

Least Privilege — Grant only required permissions.

1. Introduction

1.1 Project Contributions

1.2 Project Success Criteria

- Validated the hardened environment through a final 14/14 PASS scan.
- Produced machine-readable JSON and human-readable text evidence for the final assessment.
- Performed security remediation and architectural hardening, including private subnet migration and encrypted storage.
- Developed 14 deterministic security checks across IAM, S3, networking, secrets, CloudTrail, AI configuration and encryption.
- Implemented dynamic resource discovery to reduce dependence on hard-coded AWS resource identifiers.
- Designed a deliberately vulnerable AWS laboratory baseline for security testing.

1.3 Security Engineering Approach

The project is considered successful when the scanner can identify the targeted insecure states, the remediation steps remove those conditions, and the final scan independently confirms that all defined controls pass.

1.4 Assumptions

The project follows a defense-in-depth model. No single control is assumed to be sufficient. Identity controls reduce privilege, network controls reduce reachability, storage controls protect data, credential controls reduce secret exposure, logging supports investigation, and AI application controls address risks specific to model interaction.

1.5 Terminology

- The AI security checks in this implementation are configuration-oriented and do not represent a complete adversarial model evaluation.
- The intentionally vulnerable credential values are laboratory-only dummy values and are not real credentials.
- The security scanner is executed using an authenticated AWS identity with permission to read the configuration required by the checks.
- The AWS account used for the laboratory is dedicated or controlled for project activities.

Cloud computing has become an important foundation for modern Artificial Intelligence applications. AWS provides services that allow organizations to deploy AI workloads with scalable compute, object storage, identity management, networking, monitoring and secret management.

However, cloud infrastructure can become vulnerable when security controls are incorrectly configured. Examples include assigning excessive permissions to IAM roles, exposing EC2 instances to the public Internet, allowing unrestricted S3 access, storing credentials in configuration files and using unencrypted storage.

AI workloads introduce additional security considerations because they process potentially sensitive data and may interact with external inputs. Security therefore needs to be addressed at both the cloud infrastructure level and the application level.

This project develops a controlled AWS laboratory in which security weaknesses are deliberately introduced, detected using an automated Python scanner and subsequently remediated.

The project follows the lifecycle: Vulnerable Configuration → Security Scanning → Detection → Remediation → Rescanning → Validation.

2. Problem Statement

AWS environments can contain security vulnerabilities caused by misconfigured identity permissions, storage, networking, secrets and logging services. Manually identifying these issues across multiple AWS services is time-consuming and may result in inconsistent security assessments.

The problem addressed by this project is to design and implement an automated security assessment and hardening framework for an AWS-based AI workload that can identify common cloud security misconfigurations and verify their remediation.

3. Objectives

- Create an AWS environment representing an AI workload.
- Intentionally introduce common security vulnerabilities.
- Develop a Python/Boto3-based security scanner.
- Automatically discover project-specific AWS resources.
- Implement security checks across multiple AWS services.
- Identify excessive IAM permissions.
- Detect insecure S3 configurations.
- Identify publicly exposed network resources.
- Detect unencrypted storage.
- Identify insecure credential storage.
- Verify CloudTrail logging.
- Implement AI application security configuration checks.
- Remediate identified vulnerabilities.
- Validate the hardened environment through a final security scan.

4. Scope of the Project

The project covers Amazon EC2, Amazon VPC, Amazon S3, AWS IAM, AWS Secrets Manager, AWS CloudTrail, Amazon EBS and AI application configuration. It is a controlled configuration-assessment laboratory and does not constitute a full penetration test.

5. Background

5.1 Cloud Security

Cloud security involves protecting cloud infrastructure, applications, data and identities against unauthorized access and misuse. Important principles include least privilege, defense in depth, secure configuration, encryption, network isolation, continuous monitoring and secure credential management.

5.2 Least Privilege

Least privilege means giving an identity only the permissions required to perform its intended task. The vulnerable policy used Action:* and Resource:* to demonstrate excessive permissions. The hardened policies restrict actions and resources to operational requirements.

5.3 Network Security

Network exposure can allow unauthorized users to access cloud workloads. The project therefore evaluates Internet-exposed SSH access and public IP assignment.

5.4 Storage Encryption

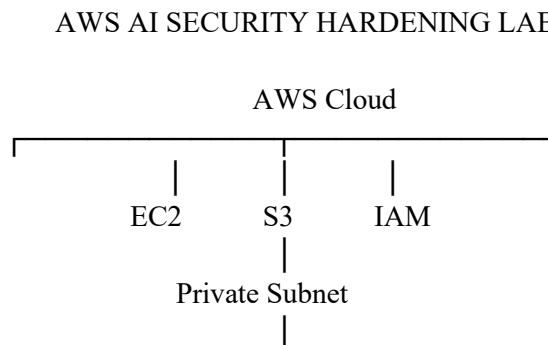
Encryption protects stored data if unauthorized access to the underlying storage occurs. The project verifies encryption of the EC2 root EBS volume.

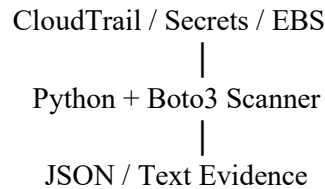
5.5 Secrets Management

Credentials should not be embedded directly into application configuration files. The vulnerable configuration demonstrates dummy credentials, while the hardened configuration references AWS Secrets Manager.

6. System Architecture

The architecture contains an AWS cloud environment, a private EC2 workload, S3 application data, IAM identities, Secrets Manager, CloudTrail and EBS encryption. A Python/Boto3 scanner discovers resources, executes security checks and generates evidence in text and JSON formats.





7. Technology Stack

Category	Technology
Cloud Platform	AWS
Programming Language	Python
AWS SDK	Boto3
Operating System	Ubuntu Linux
Virtualization	VirtualBox
Version Control	Git / GitHub
Compute	Amazon EC2
Storage	Amazon S3
Block Storage	Amazon EBS
Identity	AWS IAM
Secret Management	AWS Secrets Manager
Logging	AWS CloudTrail
Networking	Amazon VPC
Scanner Output	JSON / TXT

8. AWS Infrastructure

The project uses a VPC with CIDR 10.0.0.0/16. A dedicated private subnet with CIDR 10.0.2.0/24 was created for the hardened workload. Its route table contains only the local VPC route.

8.1 EC2

The hardened workload uses a t3.micro EC2 instance in the private subnet. It has no public IPv4 address, uses an IAM instance profile, requires IMDSv2 and has an encrypted root EBS volume.

8.2 Amazon S3

The application data bucket is protected using S3 Block Public Access, restricted application access and secure transport enforcement.

8.3 IAM

The vulnerable policy demonstrated unrestricted Action:* and Resource:* permissions. It was detached from active use and replaced with secure workload and data-access policies.

8.4 AWS Secrets Manager

The project uses the secret named AI-Security-Lab-Dummy-Credential to demonstrate secure secret management. The scanner validates the secret resource without retrieving its value.

8.5 CloudTrail

The project uses AI-Security-Lab-Trail to provide AWS activity logging and security visibility.

9. Threat Model

Threat	Potential Impact
Excessive IAM permissions	Unauthorized AWS resource access
Public SSH access	Unauthorized remote access
Public EC2 address	Increased attack surface
Public S3 access	Data exposure
Unencrypted EBS	Data confidentiality risk
Hard-coded credentials	Credential theft
Weak logging	Reduced incident visibility
Broad S3 permissions	Unauthorized data access
Weak AI input validation	Prompt/injection-related application risks
Missing output validation	Unsafe AI-generated output

10. Security Assessment Checks

ID	Security Area	Check
IAM-01	IAM	Unrestricted Action:* and Resource:*
IAM-02	IAM	Dangerous IAM actions
IAM-03	IAM	Unrestricted trust relationship
S3-01	S3	Public object access
S3-02	S3	S3 Block Public Access
S3-03	S3	Secure transport
NET-01	Network	Public SSH access
NET-02	Network	Public IPv4 address
SEC-01	Secrets	Hard-coded credentials
LOG-01	Logging	CloudTrail configuration
AI-01	AI/IAM	Unrestricted AI workload permissions
AI-02	AI/S3	Excessive AI data access
AI-03	AI Application	AI security controls
ENC-01	Encryption	EBS encryption

11. Initial Vulnerable Configuration

11.1 Excessive IAM Permissions

The vulnerable policy used Effect: Allow, Action: * and Resource: *. This violates least privilege and could allow a compromised workload to perform unauthorized actions across AWS services.

11.2 Public SSH Access

The initial security group allowed TCP/22 from 0.0.0.0/0, exposing SSH to the Internet and increasing the attack surface.

11.3 Public EC2 Exposure

The original workload had a public IPv4 address, making the instance directly reachable from the Internet.

11.4 Hard-Coded Credentials

The vulnerable configuration contained intentionally fake laboratory values for APP_PASSWORD, AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY. No real credentials were used.

11.5 AI Workload Over-Permission

The AI workload initially demonstrated broad IAM permissions and broad S3 access, illustrating the impact of excessive privileges on a data-processing workload.

12. Detection Methodology

12.1 Security Assessment Logic

12.2 Security Check Design Principles

Each security check follows a consistent evaluation pattern: identify the relevant AWS resource, retrieve only the configuration attributes required by the rule, compare the observed state with the expected secure state, record evidence, and return a PASS or FAIL result.

12.3 Expected Secure State

The scanner is designed around four principles: least privilege, secure-by-default configuration, explicit evidence and resource discovery. Checks are intended to be deterministic and understandable rather than relying on opaque risk scores.

12.4 Vulnerability-to-Control Mapping

IAM — No unrestricted Action:* / Resource:* permissions; trust policies are restricted.

S3 — Public access blocked and secure transport required.

Network — No Internet-wide SSH access and no public IPv4 address on the hardened workload.

Secrets — No active hard-coded credentials; secret is referenced through Secrets Manager.

Logging — CloudTrail exists, is actively logging, is multi-region and has an S3 destination.

AI — AI workload permissions and data access are restricted; local AI security controls are enabled.

Encryption — EC2 root EBS volume is encrypted.

12.5 Risk Prioritization

The mapping explains why each security issue matters and which control is responsible for detecting or preventing it.

The scanner automatically discovers project resources using project-specific tags and resource properties. This reduces dependence on hard-coded resource IDs and improves portability across environments.

The workflow is: Start Scanner → Create Boto3 Session → Discover Resources → Run

IAM/S3/Network/Secrets/CloudTrail/AI/Encryption Checks → Generate PASS/FAIL Results → Generate JSON/Text Evidence.

The project prioritizes vulnerabilities according to the degree to which they can increase unauthorized access, data exposure or attack surface. Critical concerns include unrestricted Action:* / Resource:* permissions. High concerns include public network exposure, broad S3 access and hard-coded credentials. Moderate concerns include missing encryption, while inadequate logging reduces investigation capability.

13. Remediation and Hardening

13.1 IAM Hardening

The unrestricted policy was removed from active use. Secure workload and data-access policies were introduced following the least-privilege principle.

13.2 Network Hardening

The unrestricted SSH rule was removed. The workload was migrated to a private subnet and the hardened instance has no public IPv4 address.

13.3 S3 Hardening

S3 Block Public Access was enabled, public object access was prevented, application access was restricted and secure transport was enforced.

13.4 Secrets Management

The vulnerable configuration was preserved separately for demonstration, while the active configuration references Secrets Manager rather than storing credentials directly.

13.5 EBS Encryption

The hardened EC2 root EBS volume was verified as encrypted.

13.6 CloudTrail

CloudTrail was configured and validated to improve visibility, accountability and incident investigation.

13.7 AI Application Controls

The AI configuration validates input handling, prompt-injection detection, output validation, sensitive-data filtering and guardrail configuration. This is a local application-level configuration check; it does not claim that an actual Amazon Bedrock Guardrail resource is deployed.

Remediation was not considered complete merely because a configuration change was made. Each change was followed by an independent scanner check. This creates a closed-loop process in which the same security rule used to detect the problem is used to verify that the problem has been removed. The original network exposure was addressed by removing unrestricted SSH access and moving the workload to a private subnet; the scanner then dynamically discovered the current instance and verified that no public IPv4 address remained. The encryption check similarly followed the EC2 root device to its EBS volume and verified the encrypted state.

13.8 Remediation Verification Strategy

14. Scanner Implementation

14.9 Evidence and Reporting

14.10 Security Scanner Execution

The project maintains two final evidence formats. The text report is intended for quick human review, while the JSON report provides structured output suitable for later automation, CI/CD integration or dashboard development. Final evidence files: `evidence/final/final-scan.txt` and `reports/final-scan.json`.

The scanner is implemented in Python using Boto3. It is organized into a central scanner, a resource discovery module and service-specific check modules.

Project structure:

```
AWS-AI-Security-Hardening-Lab/
├── scanner/
│   ├── scanner.py
│   ├── resource_discovery.py
│   ├── requirements.txt
│   └── checks/
│       ├── ai_checks.py
│       ├── iam_checks.py
│       ├── encryption_checks.py
│       ├── network_checks.py
│       ├── cloudtrail_checks.py
│       ├── secrets_checks.py
│       └── s3_checks.py
├── insecure-app-config/
│   ├── application.env
│   └── application.env.vulnerable
├── remediation/
│   └── application-secure.env
├── evidence/final/final-scan.txt
└── reports/final-scan.json
```

14.1 Resource Discovery

Project tags are used to identify lab resources such as the EC2 instance, application S3 bucket and AI IAM role.

14.2 IAM Checks

The IAM module checks unrestricted actions/resources, dangerous IAM actions and unrestricted trust principals.

14.3 S3 Checks

The S3 module checks public object access, Block Public Access settings and secure transport.

14.4 Network Checks

NET-01 checks Internet-wide SSH exposure. NET-02 checks whether the discovered lab EC2 instance has a public IPv4 address.

14.5 Secrets Check

SEC-01 searches the active application configuration for hard-coded password and AWS credential fields and validates the expected Secrets Manager resource without retrieving its value.

14.6 CloudTrail Check

LOG-01 validates trail availability, active logging, multi-region configuration and an S3 destination.

14.7 Encryption Check

ENC-01 identifies the EC2 root device and verifies that its associated EBS volume is encrypted.

14.8 AI Security Checks

AI-01 detects unrestricted AI workload permissions, AI-02 detects excessive S3 data access, and AI-03 validates local AI application security controls.

```
source .venv/bin/activate  
python -m scanner.scanner
```

The scanner is executed from the project root after activating the Python virtual environment.

15. Cross-Platform Implementation

15.1 Why Boto3 Was Used

15.2 Authentication and Credential Handling

Boto3 provides direct programmatic access to AWS services from Python. Using Boto3 allows the scanner to query AWS configuration without depending on shell commands or the AWS CLI.

15.3 Cross-Platform Considerations

The scanner relies on the AWS credential provider mechanisms available to Boto3 rather than embedding access keys in source code. The laboratory application configuration was also changed so that active configuration references Secrets Manager instead of storing credential material directly.

15.4 Operational Workflow

The scanner is written in Python and therefore can be executed on Linux, Windows or macOS, and can also be packaged into a container or CI/CD runner. The development environment used for this project was an Ubuntu Linux virtual machine running through VirtualBox.

15.5 Example Vulnerable Configuration

The practical workflow used in the laboratory can be reproduced from a clean Ubuntu terminal: activate the Python environment, confirm AWS authentication and target region, run the scanner, record failures, apply remediation, and run the scanner again to validate the hardened state.

15.6 Hardened Application Configuration

The project preserves a separate laboratory-only vulnerable file for demonstration. The values are explicitly dummy laboratory values and are retained to demonstrate hard-coded-secret detection without exposing a real credential.

15.7 Validation Philosophy

The active configuration instead references the Secrets Manager mechanism. The important security change is that the active configuration does not contain the password or AWS access key material itself; it contains a reference to the secret-management mechanism.

The project was developed on an Ubuntu Linux virtual machine running through VirtualBox. The scanner uses Python and Boto3, so it is not dependent on the operating-system-specific AWS CLI.

The scanner can be used on Linux, Windows, macOS, Docker environments or CI/CD environments where Python, Boto3 and valid AWS authentication are available.

AWS CLI was used primarily for infrastructure creation and validation. A suitable security explanation is: “AWS CLI is a cross-platform AWS API client that communicates with AWS through authenticated HTTPS API requests. Its security depends on credential management. In this project, the scanner itself uses Boto3, so it does not depend on the AWS CLI.”

A remediation is treated as successful only when the scanner observes the expected secure state. This prevents a documentation-only outcome and makes the final result measurable.

16. Final Security Validation

After remediation, the scanner was executed using:

```
source .venv/bin/activate
python -m scanner.scanner
```

Final result:

Total checks : 14

Failed : 0

Passed : 14

Check	Result
IAM-01	PASS
IAM-02	PASS
IAM-03	PASS
S3-01	PASS
S3-02	PASS
S3-03	PASS

NET-01	PASS
NET-02	PASS
SEC-01	PASS
LOG-01	PASS
AI-01	PASS
AI-02	PASS
AI-03	PASS
ENC-01	PASS

The final evidence was stored in evidence/final/final-scan.txt and reports/final-scan.json.

17. Before-and-After Comparison

Security Area	Before Hardening	After Hardening
IAM	Excessive permissions	Least privilege
IAM Trust	Potentially unrestricted	Restricted
SSH	Publicly accessible	Public SSH removed
EC2	Public IPv4	Private subnet / no public IP
S3	Potential public exposure	Block Public Access
S3 Transport	Not enforced	Secure transport enforced
Secrets	Hard-coded dummy credentials	Secrets Manager reference
AI IAM	Broad permissions	Restricted policies
AI Data Access	Broad S3 access	Required object/resource only
AI Controls	Not validated	Application controls validated
CloudTrail	Security logging requirement	Logging validated
EBS	Unencrypted volume	Encrypted volume
Scanner	Static/manual assumptions	Dynamic resource discovery
Final Scan	Vulnerabilities present	14/14 PASS

18. Security Analysis

The project demonstrates that cloud security cannot rely on a single security mechanism. Multiple layers of security controls were applied.

- Identity Layer: IAM least privilege limits what a compromised workload can do.
- Network Layer: Private subnet placement and restricted security groups reduce external attack surface.
- Data Layer: S3 controls and EBS encryption protect stored data.
- Credential Layer: Secrets Manager removes credentials from active application configuration.
- Monitoring Layer: CloudTrail provides visibility into AWS activity.
- Application Layer: AI security configuration controls provide additional protection against unsafe inputs, prompt injection and sensitive output.

19. Limitations

- The scanner evaluates a predefined set of security rules rather than every possible AWS security control.
- The project is a controlled laboratory environment rather than a production AWS workload.

- AI-03 validates application configuration rather than deploying and testing an actual Amazon Bedrock Guardrail.
- The scanner does not perform penetration testing.
- The scanner does not perform full vulnerability assessment of operating-system packages.
- Detection is limited to the AWS services and resources implemented in the project.
- The project does not currently provide continuous real-time monitoring.

20. Future Enhancements

- Integrate the scanner into GitHub Actions or another CI/CD pipeline.
- Add support for Lambda, RDS, DynamoDB, ECR, ECS, EKS, KMS and CloudWatch.
- Provide automated remediation recommendations or approved fixes.
- Introduce continuous monitoring using EventBridge, AWS Config, Security Hub or CloudWatch.
- Expand AI security testing to prompt injection, data leakage, model access control, audit logging and Bedrock Guardrails integration.

21. Conclusion

The AWS AI Security Hardening Lab successfully demonstrates the identification and remediation of common security vulnerabilities in an AWS-based AI workload.

An intentionally vulnerable AWS environment was created to simulate realistic cloud security issues involving IAM permissions, network exposure, S3 access, secrets management, encryption, logging and AI workload security. A Python-based security scanner using Boto3 was implemented to automatically discover relevant resources and perform 14 security checks.

The vulnerabilities were remediated using least-privilege IAM, private networking, restricted security groups, S3 Block Public Access, secure transport, Secrets Manager, CloudTrail, EBS encryption and AI application security controls.

The final security scan produced 14 checks passed and 0 checks failed. The project therefore demonstrates a repeatable workflow for Security Assessment → Vulnerability Detection → Remediation → Automated Validation.

22. References

1. Amazon Web Services, AWS Identity and Access Management (IAM) Documentation.
2. Amazon Web Services, Amazon S3 Security Best Practices.
3. Amazon Web Services, Amazon EC2 User Guide.
4. Amazon Web Services, Amazon VPC Documentation.
5. Amazon Web Services, AWS Secrets Manager Documentation.
6. Amazon Web Services, AWS CloudTrail User Guide.
7. Amazon Web Services, Amazon EBS Encryption Documentation.
8. Amazon Web Services, Boto3 Documentation.
9. Amazon Web Services, AWS Security Best Practices.
10. OWASP Foundation, OWASP Top 10.
11. NIST, Cybersecurity Framework.

23. Appendix

Appendix A – Project Directory

AWS-AI-Security-Hardening-Lab/

```

├── scanner/
│   ├── __init__.py
│   ├── scanner.py
│   ├── requirements.txt
│   ├── resource_discovery.py
│   └── checks/
│       ├── __init__.py
│       ├── ai_checks.py
│       ├── iam_checks.py
│       ├── encryption_checks.py
│       ├── network_checks.py
│       ├── cloudtrail_checks.py
│       ├── secrets_checks.py
│       └── s3_checks.py
├── insecure-app-config/
│   ├── application.env
│   └── application.env.vulnerable
├── remediation/
│   └── application-secure.env
├── evidence/final/final-scan.txt
├── reports/final-scan.json
└── test_aws.py

```

Appendix B – Important Commands

```

source .venv/bin/activate
python -m scanner.scanner
git status
git add .
git commit -m "Complete AWS AI security hardening and scanner"
git pull --rebase origin main
git push origin main

```

Appendix C – Recommended Screenshots

- AWS VPC configuration.
- AI-Security-Lab-Private-Subnet showing 10.0.2.0/24.
- Private route table showing only the local route.
- Hardened EC2 showing private subnet and no public IPv4 address.
- Security Group showing that TCP/22 from 0.0.0.0/0 is absent.

- Encrypted EBS volume showing Encrypted: Yes.
- IAM secure policy showing restricted permissions.
- S3 Block Public Access settings.
- Secrets Manager secret existence (never display the secret value).
- CloudTrail trail and logging status.
- Final scanner execution showing Total checks: 14, Failed: 0, Passed: 14.
- GitHub repository showing the final project structure.

Appendix D – Report Reading Note

This report is intentionally structured to make the project easy to evaluate from both an academic and technical perspective. The first sections establish the problem, objectives, scope and security concepts. The middle sections explain the AWS architecture, vulnerable baseline, detection methodology, scanner design and remediation process. The final sections provide objective validation, comparison, limitations and future work.

For a project demonstration, the recommended sequence is:

12. Begin with the problem: explain why AI workloads require layered cloud security.
13. Show the vulnerable baseline and explain one or two representative risks, such as Action:* / Resource:* and public network exposure.
14. Run the scanner and explain how resource discovery maps AWS resources to the security checks.
15. Demonstrate remediation: least-privilege IAM, private subnet, restricted security group, S3 hardening, Secrets Manager and encrypted EBS.
16. Run the final scanner and show the 14/14 PASS result.
17. Conclude by explaining that the project validates a defined security control set and is designed for future CI/CD and continuous monitoring integration.

Appendix E – Detailed Security Check Catalogue

This appendix provides a concise technical explanation of each automated check so that the report can be used as both documentation and a demonstration reference.

ID	Control	Technical Purpose	PASS Condition
IAM-01	Unrestricted IAM Permissions	The check looks for active Allow statements where Action and Resource are unrestricted. Such a policy violates least privilege because the identity is not constrained to an operational scope.	PASS when no active unrestricted Allow policy is detected.
IAM-02	Dangerous IAM Actions	The check identifies high-impact IAM capabilities that could enable privilege escalation or policy manipulation when unnecessarily granted.	PASS when the targeted dangerous actions are not unnecessarily granted.
IAM-03	Unrestricted Trust Relationship	The check examines role trust configuration for unrestricted principals. A role that can be assumed by an overly broad principal increases the risk of unauthorized role assumption.	PASS when trust is restricted.
S3-01	Public Object Access	The check evaluates whether application data can be publicly accessed through the bucket configuration.	PASS when public object access is not available.
S3-02	S3 Block Public Access	The check validates the four Block Public Access controls.	PASS when the required controls are enabled.
S3-03	Secure Transport	The check validates that insecure HTTP-style transport is denied through the SecureTransport condition.	PASS when secure transport is enforced.
NET-01	Internet-Wide SSH	The check inspects the relevant security group rules for TCP/22 from 0.0.0.0/0.	PASS when Internet-wide SSH is absent.
NET-02	Public IPv4 Address	The check dynamically identifies the lab instance and verifies whether a public IPv4 address is assigned.	PASS when no public IPv4 address is present.
SEC-01	Hard-Coded Credentials	The check scans the active application configuration for password and AWS credential fields and verifies the expected Secrets Manager resource.	PASS when hard-coded credentials are absent and the secret resource exists.
LOG-01	CloudTrail	The check validates the lab trail, active logging, multi-region configuration and S3 destination.	PASS when logging requirements are satisfied.
AI-01	AI Workload IAM	The check applies least-	PASS when unrestricted AI

AWS AI SECURITY HARDENING LAB

		privilege reasoning to the AI workload role and detects unrestricted permissions.	permissions are absent.
AI-02	AI Data Access	The check detects broad S3 access to the AI application data and expects access to be constrained to the required object/resource.	PASS when access is appropriately scoped.
AI-03	AI Application Controls	The check validates local configuration for input validation, prompt-injection detection, output validation, sensitive-data filtering and guardrails-enabled state.	PASS when all required local controls are enabled. It does not claim an actual Bedrock Guardrail deployment.
ENC-01	EBS Encryption	The check identifies the EC2 root device and verifies the associated EBS volume encryption state.	PASS when the root volume is encrypted.

Figure and Screenshot Placement

Insert actual screenshots from the AWS Console and terminal at the locations indicated below. Use captions consistently, for example “Figure 1: AWS VPC configuration used for the security hardening laboratory.”

Figure	Subject	What the screenshot should prove
Figure 1	AWS VPC	Show the 10.0.0.0/16 VPC.
Figure 2	Initial public network state	Show the original public exposure if available as evidence.
Figure 3	Private subnet	Show AI-Security-Lab-Private-Subnet and 10.0.2.0/24.
Figure 4	Private route table	Show only the local VPC route.
Figure 5	Hardened EC2	Show private subnet, no public IPv4 and IMDSv2 requirement.
Figure 6	Security Group	Show that TCP/22 from 0.0.0.0/0 is absent.
Figure 7	S3 security	Show Block Public Access and secure transport configuration.
Figure 8	IAM policy	Show the restricted secure policy.
Figure 9	Secrets Manager	Show the secret resource; never show its value.
Figure 10	CloudTrail	Show trail configuration and logging status.
Figure 11	Encrypted EBS	Show Encrypted = Yes.
Figure 12	Scanner execution	Show the terminal output with 14 checks, 0 failures and 14 passes.
Figure 13	JSON report	Show a readable excerpt from reports/final-scan.json.
Figure 14	GitHub repository	Show the final project structure and commit.

Appendix H – Reproducibility Checklist

The following checklist can be used before a final demonstration or submission.

- AWS region is set to the intended project region.
- Python virtual environment is active.
- Boto3 dependencies are installed.
- AWS authentication is available through a secure credential provider.
- Project resources are tagged consistently for discovery.
- The active application configuration contains no real credentials.
- The vulnerable configuration contains dummy values only.
- The scanner completes without unexpected exceptions.
- All 14 checks are present in the final output.
- Final evidence and JSON report are saved.
- AWS screenshots have been captured without exposing secret values.
- Temporary AWS resources retained for rollback are removed after the demonstration to avoid unnecessary cost.

APPENDIX G – IMPLEMENTATION AND SCAN EVIDENCE

The following screenshots provide visual evidence of the vulnerable baseline, remediation configuration and final security validation. They complement the human-readable and JSON scanner evidence already documented in the report.

```
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ grep -nE 'resource "aws_(vpc|subnet|security_group|iam_role|s3_b
ucket|instance|ebs_volume|cloudtrail|secretsmanager_secret)' terraform/main.tf
25:resource "aws_vpc" "lab" {
43:resource "aws_subnet" "public" {
77:resource "aws_security_group" "vulnerable" {
118:resource "aws_iam_role" "vulnerable_app" {
143:resource "aws_iam_role_policy" "vulnerable_app" {
166:resource "aws_iam_role" "at_workload" {
192:resource "aws_iam_role_policy" "at_workload_vulnerable" {
215:resource "aws_s3_bucket" "application_data" {
226:resource "aws_s3_bucket_policy" "vulnerable" {
253:resource "aws_s3_bucket_public_access_block" "vulnerable" {
267:resource "aws_secretsmanager_secret" "application" {
282:resource "aws_instance" "vulnerable" {
315:resource "aws_s3_bucket" "cloudtrail" {
324:resource "aws_cloudtrail" "lab" {
339:resource "aws_s3_bucket_policy" "cloudtrail" {
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$
```

Figure A.1 — Terraform vulnerable infrastructure definition

The Terraform configuration shows the intentionally vulnerable AWS resources used for the baseline assessment, including the VPC, public subnet, security group, IAM roles/policies, S3 resources, Secrets Manager secret, EC2 instance, and CloudTrail resources.

```
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ cat evidence/baseline/baseline-scan.txt
=====
AWS SECURITY HARDENING LAB - BASELINE SCAN
=====

[FAIL] IAM-01 | High | Excessive Application Role Permissions
Evidence: Action=* and Resource=*

[FAIL] IAM-02 | Critical | IAM Privilege Escalation Permissions
Evidence: Role contains unrestricted or IAM administrative permissions

[FAIL] IAM-03 | Critical | Overly Permissive Role Trust Relationship
Evidence: Principal allows AWS principal *

[FAIL] S3-01 | Critical | Public Application Data
Evidence: Bucket policy allows Principal=*

[FAIL] S3-02 | High | Public Access Protection Disabled
Evidence: Public Access Block configuration: {'BlockPublicAcls': False, 'IgnorePublicAcls': False, 'BlockPublicPolicy': False
, 'RestrictPublicBuckets': False}

[FAIL] S3-03 | Medium | HTTPS Not Enforced
Evidence: No aws:SecureTransport condition found

[FAIL] NET-01 | High | Internet-Exposed SSH
Evidence: TCP port 22 allows 0.0.0.0/0

[FAIL] NET-02 | High | Internet-Facing Application Workload
Evidence: EC2 instance has public IP 100.24.123.232 in subnet subnet-0009be30133f2bdc4

[FAIL] SEC-01 | High | Insecure Application Credential Storage
Evidence: Credential material found in insecure-app-config/application.env

[FAIL] LOG-01 | High | Weak CloudTrail Security Configuration
Evidence: log file validation disabled; multi-region logging disabled

[FAIL] AI-01 | Critical | Excessive Permissions for AI Workload
```

Figure A.2 — Baseline security scan: IAM, S3 and network findings

The baseline scanner reports intentionally introduced failures including excessive IAM permissions, public S3 exposure, disabled S3 Public Access Block controls, missing HTTPS enforcement, and Internet-wide SSH exposure.

```

Evidence: Bucket policy allows Principal=*
[FAIL] S3-02 | High | Public Access Protection Disabled
Evidence: Public Access Block configuration: {'BlockPublicAcls': False, 'IgnorePublicAcls': False, 'BlockPublicPolicy': False, 'RestrictPublicBuckets': False}
[FAIL] S3-03 | Medium | HTTPS Not Enforced
Evidence: No aws:SecureTransport condition found
[FAIL] NET-01 | High | Internet-Exposed SSH
Evidence: TCP port 22 allows 0.0.0.0/0
[FAIL] NET-02 | High | Internet-Facing Application Workload
Evidence: EC2 instance has public IP 100.24.123.232 in subnet subnet-0009be30133f2bdc4
[FAIL] SEC-01 | High | Insecure Application Credential Storage
Evidence: Credential material found in insecure-app-config/application.env
[FAIL] LOG-01 | High | Weak CloudTrail Security Configuration
Evidence: log file validation disabled; multi-region logging disabled
[FAIL] AI-01 | Critical | Excessive Permissions for AI Workload
Evidence: AI workload role AI-Security-Lab-AI-Workload-Role has Action=* and Resource=*
[FAIL] AI-02 | High | Excessive Data Access by AI Workload
Evidence: AI workload has S3 access to ai-security-lab-data-256148542278
[FAIL] AI-03 | High | Missing AI Input/Output Security Controls
Evidence: Disabled AI security controls: input_validation, prompt_injection_detection, output_validation, sensitive_data_filtering, guardrails_enabled
=====
Total checks : 13
Failed       : 13
Passed      : 0
=====
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ grep -nE 'resource "aws_(vpc|subnet|security_group|iam_role|s3_bucket|instance|efs_volume|cloudtrail|secretsmanager_secret)' terraform/main.tf

```

Figure A.3 — Baseline security scan: secrets, logging and AI findings

The baseline scan continues with failures for public IPv4 exposure, insecure credential storage, weak CloudTrail configuration, excessive AI workload permissions, excessive AI data access, and missing AI input/output security controls.

```

sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ find remediation -maxdepth 1 -type f | sort
remediation/al-data-access-secure-policy.json
remediation/al-data-access-vulnerable-policy.json
remediation/al-workload-secure-policy.json
remediation/al-workload-trust-policy.json
remediation/al-workload-vulnerable-policy.json
remediation/application-secure.env
remediation/secure-bucket-policy.json
remediation/secure-policy.json
remediation/secure-trust-policy.json
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ cat remediation/application-secure.env
APP_USERNAME=demo-app
APP_PASSWORD_FROM_SECRETS_MANAGER=true
AWS_CREDENTIALS_FROM_SECRETS_MANAGER=true
SECRET_NAME=AI-Security-Lab-Dummy-Credential
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ cat remediation/secure-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadApplicationSecret",
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:us-east-1:256148542278:secret:AI-Security-Lab-Dummy-Credential-*"
    }
  ]
}
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$

```

Figure A.4 — Remediation configuration

The remediation evidence shows the secure application configuration and least-privilege Secrets Manager policy used to remove hard-coded credentials and restrict secret access.

```
(.venv) sthuthi@sthuthi-VirtualBox:~/AWS-AI-Security-Hardening-Lab$ python -m scanner.scanner

=====
AWS SECURITY HARDENING LAB - SECURITY SCAN
=====

[PASS] IAM-01 | High | Excessive Application Role Permissions
Resource: AI-Security-Lab-Vulnerable-Role
Evidence: No unrestricted Action/Resource combination found in attached policies.
Impact: The application role is protected from unrestricted AWS permissions.
Remediation: Continue enforcing least-privilege IAM permissions.

[PASS] IAM-02 | Critical | IAM Privilege Escalation Permissions
Resource: AI-Security-Lab-Vulnerable-Role
Evidence: No unrestricted IAM permissions detected in attached policies.
Impact: The application role does not contain unrestricted or detected dangerous IAM permissions.
Remediation: Continue enforcing least-privilege IAM permissions.

[PASS] IAM-03 | Critical | Overly Permissive Role Trust Relationship
Resource: AI-Security-Lab-Vulnerable-Role
Evidence: Trust relationship is restricted.
Impact: The application role can only be assumed by its intended trusted principal.
Remediation: Continue restricting the trust relationship to required principals only.

[PASS] S3-01 | Critical | Public Application Data
Resource: ai-security-lab-data-256148542278
Evidence: No public Allow statement detected for ai-security-lab-data-256148542278.
Impact: Application data is not publicly accessible through the bucket policy.
Remediation: Continue restricting bucket access to authorized principals.

[PASS] S3-02 | High | Public Access Protection Disabled
Resource: ai-security-lab-data-256148542278
Evidence: All four Public Access Block controls are enabled for ai-security-lab-data-256148542278.
Impact: All S3 Public Access Block controls are enabled, reducing the risk of accidental public exposure.
Remediation: Continue keeping all four Public Access Block controls enabled.

[PASS] S3-03 | Medium | HTTPS Not Enforced
Resource: ai-security-lab-data-256148542278
Evidence: Bucket policy for ai-security-lab-data-256148542278 contains an aws:SecureTransport condition.
```

Figure A.5 — Final scan: IAM, S3 and network validation

The hardened environment passes the IAM, S3 and network security controls, demonstrating removal of unrestricted permissions, public S3 exposure and Internet-wide SSH access.

```
[PASS] S3-03 | Medium | HTTPS Not Enforced
Resource: ai-security-lab-data-256148542278
Evidence: Bucket policy for ai-security-lab-data-256148542278 contains an aws:SecureTransport condition.
Impact: S3 requests are protected against insecure non-HTTPS transport.
Remediation: Continue enforcing aws:SecureTransport for application bucket access.

[PASS] NET-01 | High | Internet-Exposed SSH
Resource: sg-032c8c60db89d1c98
Evidence: Security group sg-032c8c60db89d1c98 does not allow TCP port 22 from 0.0.0.0/0.
Impact: SSH is not exposed to the entire IPv4 internet.
Remediation: Continue restricting administrative access to trusted sources.

[PASS] NET-02 | High | Internet-Facing Application Workload
Resource: i-0f3751859f83badca
Evidence: EC2 instance i-0f3751859f83badca has no public IP. Private IP: 10.0.1.148, subnet: subnet-0009be30133f2bdc4.
Impact: The EC2 workload does not have a public IP address, reducing its direct internet exposure.
Remediation: Continue keeping the application workload private and avoid unnecessary public IP assignments.

[PASS] SEC-01 | High | Insecure Application Credential Storage
Resource: AI-Security-Lab-Dummy-Credential
Evidence: Application configuration uses AWS Secrets Manager secret AI-Security-Lab-Dummy-Credential.
Impact: Application credentials are no longer stored directly in the application configuration. Using AWS Secrets Manager reduces the risk of credential exposure.
Remediation: Store application credentials in AWS Secrets Manager and retrieve them at runtime instead of hardcoding credential values in application configuration files.

[PASS] LOG-01 | High | Weak CloudTrail Security Configuration
Evidence: Log validation and multi-region logging enabled

[PASS] AI-01 | Critical | Excessive Permissions for Generative-AI Workload
Resource: AI-Security-Lab-AI-Workload-Role
Evidence: No unrestricted Action/Resource combination found in the AI workload policies.
Impact: The AI workload is protected against unrestricted AWS permissions.
Remediation: Continue enforcing least-privilege IAM permissions.

[PASS] AI-02 | High | Excessive Data Access by Generative-AI Workload
Resource: ai-security-lab-data-256148542278
Evidence: AI workload data access is restricted to the required application resource.
```

Figure A.6 — Final scan: secrets, logging and AI validation

The hardened environment passes secrets management, CloudTrail and Generative-AI security checks. AI data access is restricted to the required application resource.

```

Remediation: Store application credentials in environment variables instead of hardcoding credentials in application configuration files.

[PASS] LOG-01 | High | Weak CloudTrail Security Configuration
Evidence: Log validation and multi-region logging enabled

[PASS] AI-01 | Critical | Excessive Permissions for Generative-AI Workload
Resource: AI-Security-Lab-AI-Workload-Role
Evidence: No unrestricted Action/Resource combination found in the AI workload policies.
Impact: The AI workload is protected against unrestricted AWS permissions.
Remediation: Continue enforcing least-privilege IAM permissions.

[PASS] AI-02 | High | Excessive Data Access by Generative-AI Workload
Resource: ai-security-lab-data-256148542278
Evidence: AI workload data access is restricted to the required application resource.
Impact: The AI workload is restricted to the minimum required application data.
Remediation: Maintain least-privilege access to arn:aws:s3::ai-security-lab-data-256148542278/application-data.txt.

[PASS] AI-03 | High | Missing Generative-AI Input/Output Security Controls
Resource: insecure-app-config/ai-config.json
Evidence: All required generative-AI input/output security controls are enabled.
Impact: The generative-AI application has configured input/output security protections.
Remediation: Maintain input validation, prompt-injection detection, output validation, sensitive-data filtering, and guardrail controls.

[PASS] ENC-01 | High | Unencrypted EBS Volume
Resource: vol-069076c670e9b82fd
Evidence: EBS volume vol-069076c670e9b82fd attached to instance i-0f3751859f83badca is encrypted.
Impact: The workload root volume is protected using encryption at rest.
Remediation: Maintain encryption for the workload's EBS storage.

=====
Total checks : 14
Failed       : 0
Passed      : 14
=====

(.venv) sthuthi@sthuthi-VirtualBox: ~/AWS-AI-Security-Hardening-Lab$

```

Figure A.7 — Final scan: encryption and overall result

The final scanner verifies encrypted EBS storage and reports 14 total checks, 0 failures and 14 passes.